Design of an AXI4-Lite I2S Transmitter IP for a MicroBlaze V SoC Using Verilog

Muhammad Muqtada Alhaddad
Department of Electrical and Information Engineering
Universitas Gadjah Mada
Yogyakarta, Indonesia
muhammadmuqtadaalhaddad@mail.ugm.ac.id

Agus Bejo
Department of Electrical and Information Engineering
Universitas Gadjah Mada
Yogyakarta, Indonesia
agusbj@ugm.ac.id

Wahyu Dewanto
Department of Electrical and Information Engineering
Universitas Gadjah Mada
Yogyakarta, Indonesia
wahyud@ugm.ac.id

***Abstract*—This paper presents the implementation of a custom I2S transmitter IP core in Verilog that is controlled through an AXI4-Lite register interface on a MicroBlaze V soft-core SoC. The target platform is a Basys3 FPGA board driving an external PCM5102A DAC. Audio timing is generated from a 100 MHz system clock using a numerically controlled oscillator, while the firmware streams stereo samples through memory-mapped register writes and status polling. The final design supports Philips I2S framing, 32-bit slots with 24-bit audio, and programmable integer sampling rates without relying on FIFO, DMA, or a licensed audio IP block. Simulation and hardware validation confirm correct waveform generation, stable audio output, and a compact implementation suitable as reusable soft IP.**

***Index Terms*—I2S, AXI4-Lite, MicroBlaze V, FPGA, Verilog, PCM5102A, soft IP**

I. Introduction

Digital audio on FPGA is attractive when deterministic timing, configurable interfaces, and software-controlled peripheral access are required. The I2S protocol remains a practical transport for PCM audio because it is simple, standardized, and directly compatible with external DACs [1]. For SoC control, AXI4-Lite provides a compact memory-mapped interface for custom peripherals [2].

This work follows the thesis implementation path for a custom I2S transmitter IP that is integrated into a MicroBlaze V based SoC on Basys3 and connected to a PCM5102A DAC [3]–[5]. The goal is not only to produce correct I2S waveforms, but also to keep the peripheral reusable as a soft IP block in Vivado designs [6].

The main contribution is a complete hardware-software flow: a Verilog I2S transmitter core, an AXI4-Lite register map, an internal phase-accumulator timing generator, and bare-metal firmware that streams audio samples using status polling. The implementation stays transparent at RTL level and avoids dependence on a licensed vendor audio block.

II. Background and Related Concepts

A. I2S audio framing

I2S uses three signals: bit clock, word select, and serial data. Philips I2S requires a one-bit delay between the word-select transition and the first data bit, and the design in this paper preserves that rule [1]. The transmitter operates in stereo mode with 32-bit slots, while the active sample width is 24 bits.

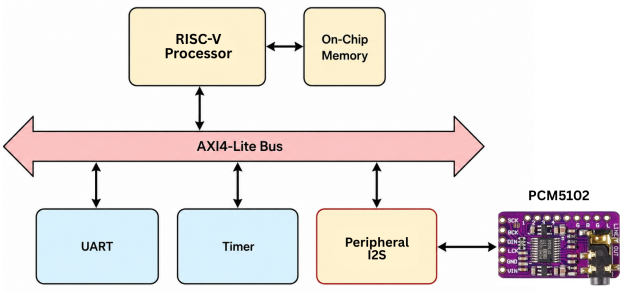


Fig. 1. System architecture linking MicroBlaze V, UART, AXI4-Lite, and the custom I2S transmitter IP.

B. AXI4-Lite register access

AXI4-Lite is used as a simple register-access protocol for control and status traffic [2], [7]. In this design, the software writes audio samples and control settings to memory-mapped registers, then waits for a status transition that indicates the hardware has latched the sample pair.

C. Clock generation

The audio timing is derived from a 100 MHz system clock using a numerically controlled oscillator. NCO-style phase accumulation is a standard way to generate programmable output frequencies in digital systems [8], [9]. Here, that technique is used to derive BCLK and MCLK without requiring a separate audio PLL.

III. System Design

A. Hardware architecture

The final system contains four main parts: MicroBlaze V, AXI interconnect, the custom I2S IP, and the external PCM5102A DAC. The processor writes to the I2S peripheral through AXI4-Lite, and the peripheral outputs `i2s_mclk`, `i2s_bclk`, `i2s_ws`, and `i2s_data`. UART is reserved for debug output during development.

TABLE I
AXI4-LITE REGISTER MAP OF THE I2S TRANSMITTER

Register	Offset	Function
DATA_LEFT	0x00	Left-channel audio sample written by software
DATA_RIGHT	0x04	Right-channel audio sample written by software
CONTROL	0x08	Enable, mute, and programmable sampling-rate control
STATUS	0x0C	Frame-latch status bit used for software synchronization

B. Register map

Table I summarizes the peripheral interface.

The control path is intentionally small so the firmware can interact with the peripheral using ordinary load/store operations rather than a driver stack. This keeps the software side suitable for bare-metal Vitis projects.

C. Clock and frame timing

The implementation uses a 100 MHz reference clock and generates the audio clocks from phase accumulators. The design aligns the channel boundary with the 64-bit stereo frame, meaning one frame contains 32 BCLK periods for the left channel and 32 for the right channel. A status toggle marks the moment when the hardware has moved the current sample pair into its internal shadow registers.

IV. IMPLEMENTATION

A. RTL structure

The RTL is organized as a soft IP core written in Verilog [10]. The core contains three functional blocks: AXI4-Lite handshake logic, a register file, and an I2S serializer. The register file stores the audio samples and control bits, while the serializer emits the Philips I2S bit stream. The clock generator and frame counter are kept inside the core so the design is self-contained.

B. Firmware flow

The bare-metal firmware initializes the peripheral, writes the control word, waits for the frame-status transition, and then pushes the next stereo sample pair. The software uses status polling instead of delays or interrupts so that sample submission stays synchronized with the frame boundary.

This approach avoids the drift that would occur if the software tried to approximate audio timing using sleep calls alone. It also keeps the transfer path deterministic across sample-rate changes.

C. Supported operating modes

The implementation supports the following modes:

- Philips I2S framing with one-bit delay.
- Stereo output with 32-bit slots.
- 24-bit active sample width.
- Programmable integer sampling rates, including standard audio rates such as 44.1 kHz, 48 kHz, 88.2 kHz, and 96 kHz.

Resource	Utilization	Available	Utilization %
LUT	2435	20800	11.71
LUTRAM	107	9600	1.11
FF	2330	41600	5.60
BRAM	32	50	64.00
IO	8	106	7.55

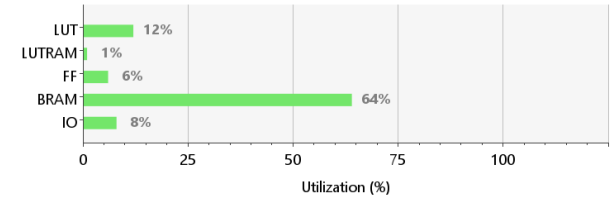


Fig. 2. Resource utilization summary of the implemented SoC and I2S transmitter.

TABLE II
REPORTED IMPLEMENTATION SUMMARY

Metric	Value
Slice registers and logic	903 Slice
LUT usage	2435 LUT
Flip-flop usage	2330 FF
BRAM usage	32 BRAM
Total power	0.209 W
WNS at 100 MHz	0.064 ns
Estimated Fmax	100.64 MHz

- Mute and enable control through the AXI4-Lite control register.

The complete source code, including the Verilog RTL, AXI4-Lite peripheral, firmware, and Vivado project, is available in the accompanying GitHub repository [11].

V. RESULTS AND DISCUSSION

A. Functional validation

The RTL was verified in simulation before board deployment. The checks covered AXI4-Lite read/write behavior, BCLK and WS generation, one-bit I2S delay, stereo frame alignment, and mute behavior. After the RTL passed simulation, the bitstream was implemented on Basys3 and the DAC output was observed on the physical board.

Figure 1 summarizes the full hardware-software chain used in the implementation.

The hardware test confirmed that the PCM5102A produced clean audio output for the intended modes. In particular, the final firmware avoided the sign-bit corruption and timing drift issues that were encountered during development.

B. Synthesis summary

Table II summarizes the reported implementation results.

These numbers indicate that the design is lightweight enough for Artix-7 class devices while still providing a reusable audio peripheral and software control path.

Figure 2 provides a compact view of the implementation footprint reported by the thesis build.

C. Discussion

The main advantage of the final architecture is that the peripheral does not depend on a third-party audio IP license. The core remains transparent at RTL level, and the register map is simple enough to be driven from bare-metal firmware. Compared with a FIFO- or DMA-based design, this implementation is easier to understand and sufficient for modest audio streaming workloads.

The tradeoff is that software must respect the frame boundary and keep up with the status polling loop. That is acceptable for the intended use case, but a higher-throughput system would benefit from FIFO buffering or DMA support.

VI. Conclusion

A custom I2S transmitter IP was successfully designed in Verilog and integrated into a MicroBlaze V SoC using AXI4-Lite control. The design generates Philips I2S timing, supports stereo 24-bit audio in 32-bit slots, and uses a phase-accumulator clocking scheme to support programmable integer sampling rates. Hardware validation with a PCM5102A DAC and synthesis results on Basys3 show that the implementation is compact, reusable, and practical for FPGA-based audio experiments.

Future work can extend the core with TX FIFO buffering, interrupt-driven transfers, or DMA support for higher sustained throughput.

References

[1] Philips Semiconductors, "I^{2}S bus specification," Philips Semiconductors, Tech. Rep., Feb. 1986, revision 1. Tersedia: https://www.nxp.com/docs/en/user-guide/UM11732.pdf.

[2] ARM Limited, "AMBA AXI and ACE protocol specification," ARM Limited, Tech. Rep. ARM IHI 0022D, 2010, issue D. Tersedia: https://developer.arm.com/documentation/ihi0022/d/.

[3] Advanced Micro Devices, Inc., *MicroBlaze Processor Reference Guide*, Advanced Micro Devices, Inc., 2021, tersedia: https://www.xilinx.com/support/documentation/sw_manuals/xilinx2021_2/ug984-vivado-microblaze-ref.pdf.

[4] Digilent Inc., *Basys 3 FPGA Board Reference Manual*, Digilent Inc., 2020, tersedia: https://digilent.com/reference/programmable-logic/basys-3/reference-manual.

[5] Texas Instruments, "PCM5102A 2-VRMS, 112-dB audio stereo DAC with PLL and 32-bit, 384-kHz PCM interface," Texas Instruments, Tech. Rep. SLAS764B, 2014, datasheet. Tersedia: https://www.ti.com/lit/ds/symlink/pcm5102a.pdf.

[6] Advanced Micro Devices, Inc., *Vivado Design Suite User Guide: Designing with IP*, Advanced Micro Devices, Inc., 2021, tersedia: https://www.xilinx.com/support/documentation/sw_manuals/xilinx2021_2/ug896-vivado-ip.pdf.

[7] ——, *AXI Reference Guide*, Advanced Micro Devices, Inc., 2017, tersedia: https://www.xilinx.com/support/documentation/ip_documentation/axi_ref_guide/latest/ug1037-vivado-axi-reference-guide.pdf.

[8] J. Vankka and K. A. I. Halonen, *Direct Digital Synthesizers: Theory, Design and Applications*. Boston: Kluwer Academic Publishers, 2001.

[9] G. P. Ao, "The basic principle and FPGA implementation of NCO," in *Proceedings of the 2012 Second International Conference on Instrumentation, Measurement, Computer, Communication and Control*. Yantai, China: IEEE, 2012.

[10] S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed. Prentice Hall, 2003.

[11] M. M. Alhaddad, "thesi2s: I2s transmitter axi4-lite ip for microblaze v," https://github.com/AdakHaddad/thesi2s, 2026, gitHub repository. Accessed July 2026.